Facts and sources: hardening plan

> **Status, 6 October 2026.** A cut-down Phase 1 is built: the per-turn sources record (`src/shared/sources.ts`, `Turn.sources`), the marks on cited links, the notice ahead of the next message and the rule in the agent block (README, "Sources"). Dropped from it: blocked-page detection, the journal entry (the notice no longer depends on self-check mode) and the turn-head count. Phases 2 to 4 are not started; the eval scenarios should be rewritten to use fixture pages, since live sites make the pass rate drift with the web.

What went wrong with fact-finding on 6 October 2026, how a research turn flows through Rocket today, and the changes that make "did the model actually read that page" a thing the app checks, shows and measures, instead of something the user has to catch.

What happened

In one chat a model answered questions about vendor policy, partner programs and product limits. Three things went wrong at once:

1. The chat's web search tool returns an AI-written digest of result pages. The model treated the digest as the source and cited the digest's URLs as if it had opened them.
2. The primary pages (openai.com, help.openai.com, x.com) were behind bot checks, so the model fell back to blogs and news sites and presented their claims with the same confidence.
3. Nothing in Rocket distinguishes a page the model opened from one that merely appeared in a result list, so nothing could flag the difference, in the reply or afterwards.

The user stopped the work. One claim they challenged (the ChatGPT desktop app's Linux status) turned out to be right, but it had been sourced from news sites; it took four more fetches to reach OpenAI's own page. That is the shape of the problem: the model can be right by luck and wrong by the same process, and the app cannot tell which.

How a research turn reaches the screen today

- **Rocket's own browser tools** (`src/host/tools/browser.ts`: `browser_read`, `browser_run`, `browser_download`, `browser_screenshot`) go through the tool registry, whose observer is the tool ledger (`src/host/tool-ledger.ts`). The ledger keeps each call's name, arguments (`safeEvidence`) and outcome in a SQLite profile database for 30 days. A `browser_read` call's `url` argument is in there; the navigations a `browser_run` script makes are not, because they happen inside the script worker as `goto` / `open` / `read` ops to main (`BrowserAgentOp` in `src/shared/types.ts`).
- **Provider-native web tools** (Claude's `WebFetch` and `WebSearch`, Codex's `webSearch`, Cursor's `fetch`, Antigravity's web tools) never touch the registry. Each adapter turns them into transcript items of `kind: 'web'` whose `label` is the URL or the query (`describeTool` in `src/host/providers/claude.ts:877`, `codex.ts:492`, `cursor.ts:446`, `antigravity.ts:504`). The label is the only trace, and a fetch and a search look the same kind.
- **The reply** is an `assistant` item. `ChatMarkdown.tsx` renders its links through `openChatLink`, with no knowledge of what the turn opened.

- **The journal** (`src/host/journal.ts`) records errors, slow turns and model notes, and produces the `<self-check>` line ahead of the next message when there is trouble. It has no notion of citations.
- **Evals** grade a turn export (`src/host/eval-export.ts`, `TurnExport.items`) with the graders in `src/host/eval-grade.ts`: `turn_completed`, `files_unchanged`, `tool_error_codes` and so on. Nothing grades what was read or cited.

So the raw material exists in two places (ledger arguments, `web` items) but is never joined, never compared with the reply, and never shown.

The plan

Four phases. The first two are deterministic and cheap. The third makes the problem measurable. The fourth uses a model and waits until the third can say whether it helps.

Phase 1: a sources record per turn, and a citation check at turn end

Collect what was opened. A new `src/host/sources.ts` keeps, per live turn, a list of `{ url, via, status, at }`:

- `via: 'browser'`: every `BrowserAgentOp` with a URL that passes through the browser tool's `BrowserLink.request` (`open`, `goto`, `read` with a url, `fetch`, `download`), plus the tab's final URL from the op's result when main reports it, so a script's `tab.goto` counts. The browser tools already hold the `BrowserLink`; wrapping it is one place.
- `via: 'provider'`: every `web` tool item whose label parses as a URL, taken from the hub's `putItem` path when the item reaches `status: 'done'`. A label that is not a URL is a search query and is recorded under `searches`, not `opened`. Adapters stay ignorant of this: the hub reads items it already receives.
- `status`: `ok`, `blocked` or `failed`. Blocked is detected from the result, not guessed: an HTTP 403, 451 or 429 from a fetch, or a read whose title is a bot-check page ("Just a moment...", "Attention Required", "Access denied") or whose text is under a small threshold. The exact list lives in one function with a test beside it.

Compare with the reply. In `Hub.endTurn` (`src/host/hub.ts:609`), when the turn completed, extract every URL from the final assistant text (markdown links and bare URLs), normalise both sides (lowercase host, drop fragment, drop `utm_*` and tracking params, drop a trailing slash, treat `.md` and `.html` twins of the same path as one page), and classify each cited URL as `opened`, `blocked` (attempted, never read) or `unopened`. The result goes on the turn:

```
// src/shared/types.ts
export type TurnSources = {
  opened: Array<{ url: string; via: 'browser' | 'provider'; status: 'ok' | 'blocked' | '
  searches: string[]
  cited: Array<{ url: string; state: 'opened' | 'blocked' | 'unopened' }>
}
```

stored by `store.ts` with the turn and sent to the window in the turn's snapshot.

Tell the next turn. When a completed turn cites an unopened or blocked URL, the hub writes a journal entry (`source: 'turn'`, `kind: 'citation'`, `severity: 'warn'`, the URLs in `details`). The

existing `<self-check>` nudge then carries it ahead of the next message, so the model is told in its own words that it cited pages it never read, and `rocket_diagnostics` lists them.

Show it. Mockup first (`docs/mockups/sources.html`), then:

- each link in a reply gets a mark: a tick for opened, a warning for blocked, a cross for unopened, with the state in the title attribute;
- the turn head gets a count, "4 sources, 1 not opened", that opens a small list of what the turn opened, searched and cited.

Rules. No model judgment anywhere in this phase. A URL counts as opened only when the app saw it leave for the page. A search result is never a source. Two pages are the same only by host and path, never by content.

Files. `src/host/sources.ts` and `sources.test.ts`; `src/host/tools/browser.ts` (observe the link); `src/host/hub.ts` (record `web` items, run the check in `endTurn`); `src/host/store.ts` and `src/shared/types.ts` (`TurnSources` on the turn); `src/host/journal.ts` (the `citation` kind in the nudge's counts); `src/renderer/src/components/ChatMarkdown.tsx` and `Transcript.tsx` (marks and the turn-head count); README's Self-check section.

Phase 2: say the rule where the model reads it

- `src/host/agent-situation.ts` gets one short paragraph: a search result is a lead, not a source; cite only pages opened this turn; when a primary page is blocked, say the fact is unverified rather than citing a page that quotes it; state facts in two tiers, verified from an opened page or reported by a named outlet and not verified.
- The handoff (`src/host/handoff.ts`) passes a turn's `cited` list with its states to a provider that takes over a chat, so a second provider does not inherit unverified claims as settled.
- Optional, and deferred: a Rocket-owned `web_search` tool that returns titles, URLs and snippets only, with no digest, and the Claude adapter's `disallowedTools` set to drop the built-in `WebSearch`. Not in scope until Phase 3 shows the prompt line is not enough, since it changes what every provider sees and costs a tool schema in every prompt.

Phase 3: research eval scenarios

A new scenario file `docs/evals/scenarios/sources.jsonl` with `runsAlone: true` (they share the one browser) and five rows, each a question whose answer lives on a known primary page:

1. A question answered on an official docs page that opens cleanly (OpenAI's Codex app-server transports). Expect: opened the primary host, cited only opened pages.
2. A question whose primary page is behind a bot check (an openai.com policy page). Expect: the reply labels the fact as unverified, cites no page it did not open.
3. A question with a tempting, widely-syndicated secondary answer (the ChatGPT Linux app's computer-use status). Expect: the primary host was opened before the reply.
4. A question with no reliable source at all (user counts for a one-month-old product). Expect: the reply says so, and cites nothing unopened.
5. A question answered by this repo's own adapter headers. Expect: the file was read, no web fetch was needed.

Each row carries `expect: { primaryHosts: string[] }`. `EvalScenario` in `src/host/eval-scenarios.ts` gains that field, and `TurnExport` gains the turn's `sources`. Three new graders in

`src/host/eval-grade.ts`, each with a test:

- `sources_primary_opened`: at least one `opened` URL with status `ok` is on a primary host.
- `citations_opened`: every cited URL is `opened`; a `blocked` or `unopened` citation fails.
- `unverified_labeled`: when any attempted source was `blocked`, the reply contains an unverified marker ("not verified", "reported by", "could not open").

Run across Claude and Codex, three trials, and keep the pass rate on the scenario page. This is the number that says whether the Phase 2 prompt line was enough, and later whether Phase 4 adds anything. Running it spends subscription usage, so it is started by hand, never by a check.

Phase 4: a second model checks the claims (later)

Goals already have a `check` role: a second provider judges a round against a checklist (`src/host/goals.ts`). The same shape fits research replies: the checker receives the reply, the turn's `opened` list, and the ledger's stored page text for each opened URL (capped), and returns one verdict per factual claim: `supported by <url>`, `contradicted by <url>`, or `not in any opened page`. Claims in the last group are marked in the transcript.

Off by default, switched on per chat. Built only after Phase 3 has a baseline, so the extra turn's cost can be weighed against a measured change in pass rate. It never replaces Phase 1: a model saying a claim is supported is still a model.

Prerequisite from the other chat

Primary pages have to open. openai.com, help.openai.com and x.com failed with bot checks or empty pages during the session that prompted this plan. The browser-realism and cookie-import work in progress is what removes the pressure toward secondary sources; the citation check then catches whatever pressure remains.

Checks

- `npm run typecheck` and `npm test` after each phase; new tests sit beside `sources.ts`, `eval-grade.ts` and the journal nudge.
- `npm run check:ui -- --visible` for the link marks and the turn-head count, with a scenario in `tests/ui/` that replays a turn whose reply cites one opened and one unopened URL.
- Phase 3 runs are live turns; ask before each run.

Open questions

- URL equivalence: `.md` twins of docs pages are the same page, but `?page=2` is not. The normaliser needs a short list of query parameters that change the page; start with none and add from real cases.
- Codex's `webSearch` item carries only the query, and Codex has no separate fetch item. If Codex reads a page inside its own search tool, that read is invisible. Verify against the app-server schema and record it in the adapter header before counting Codex searches as anything.
- Whether to keep the opened page's text for Phase 4 in the ledger (already redacted and capped by `safeEvidence`) or refetch at check time. Keeping it means the checker sees what

the model saw.